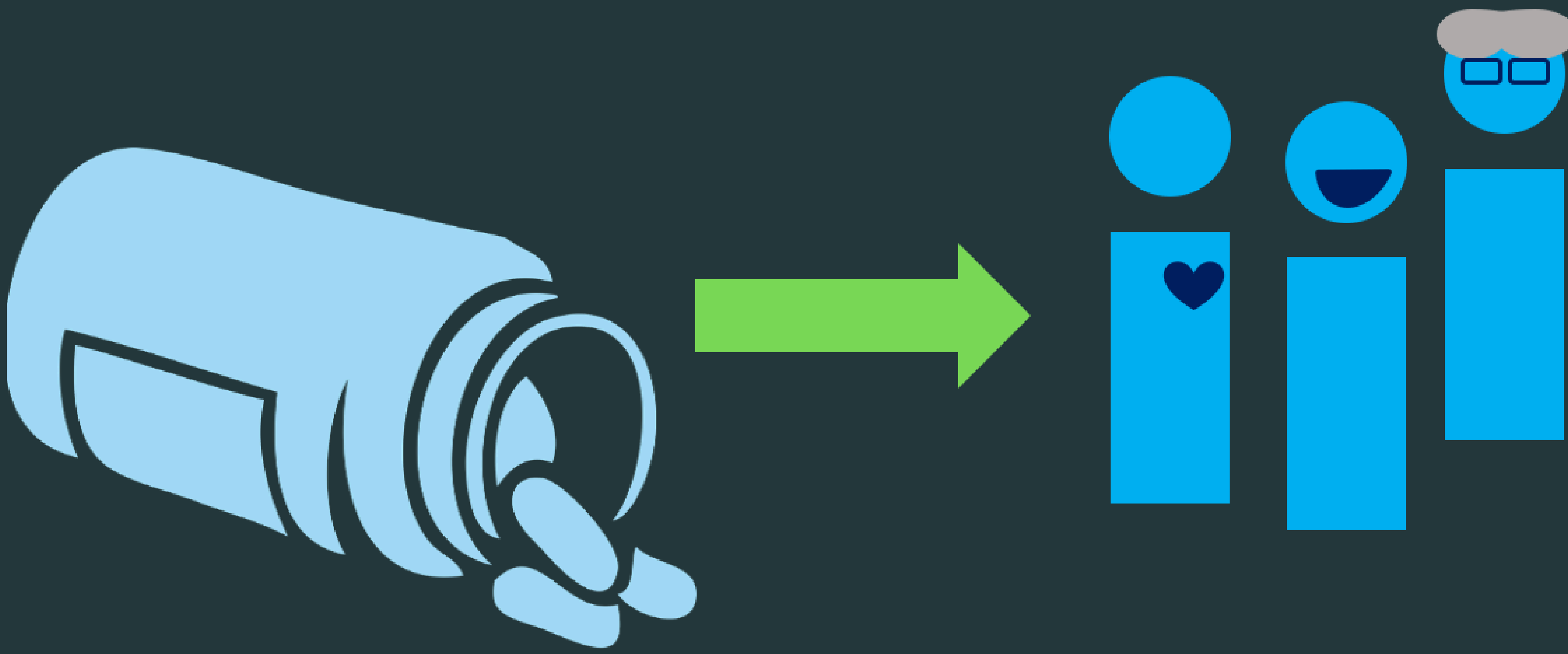
# Causal Inference with `group_by()` and `summarize()`

Lucy D'Agostino McGowan

Wake Forest University

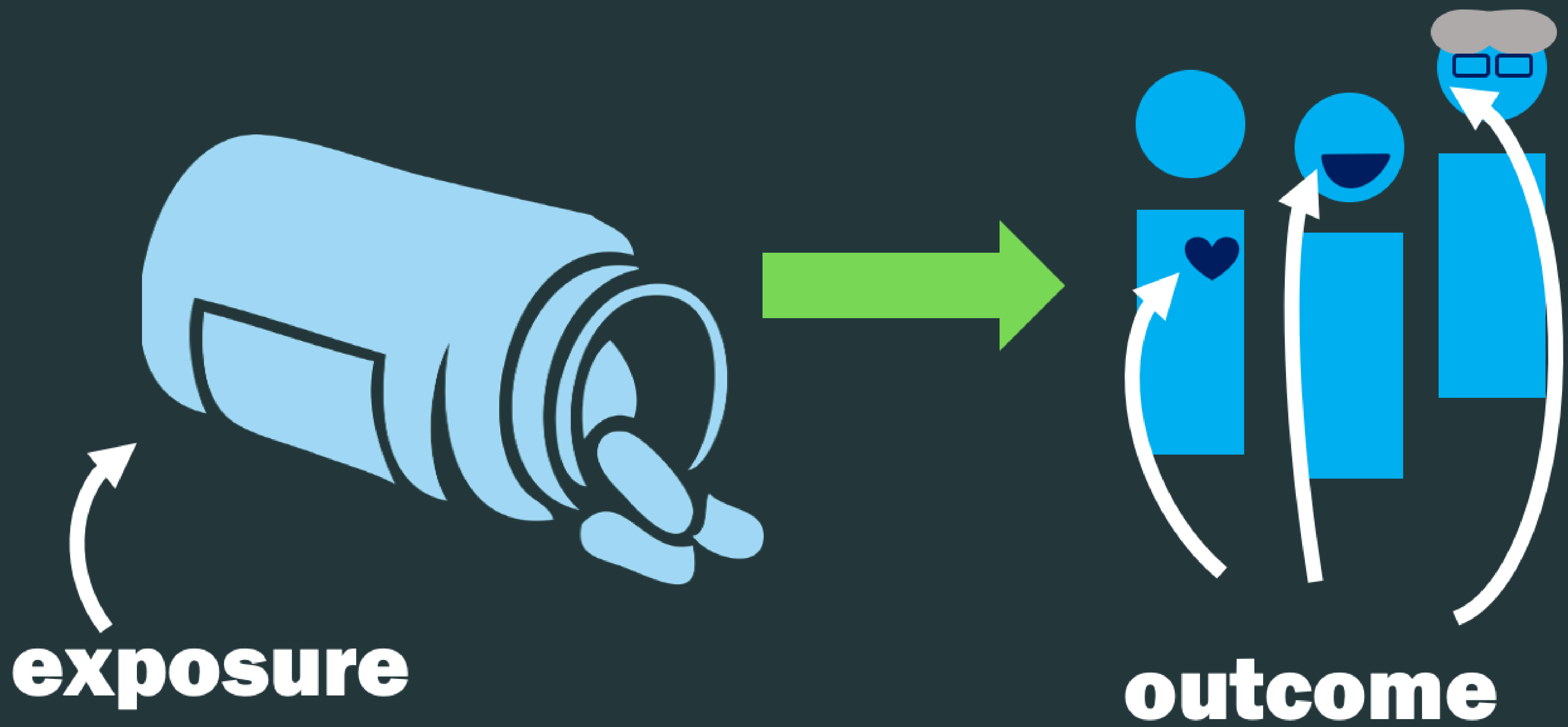
# Observational Studies

**Goal**: To answer a research question



# Observational Studies

**Goal**: To answer a research question



# Observational Studies

## Randomized Controlled Trial

# Observational Studies

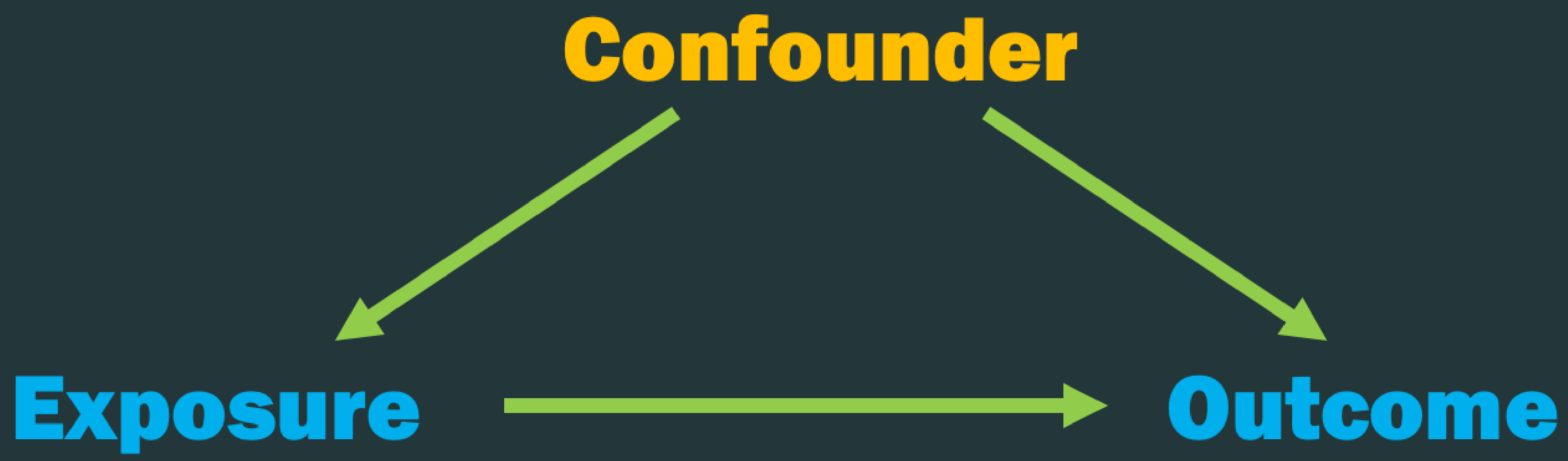
## Randomized Controlled Trial


**Treatment** P=0.5

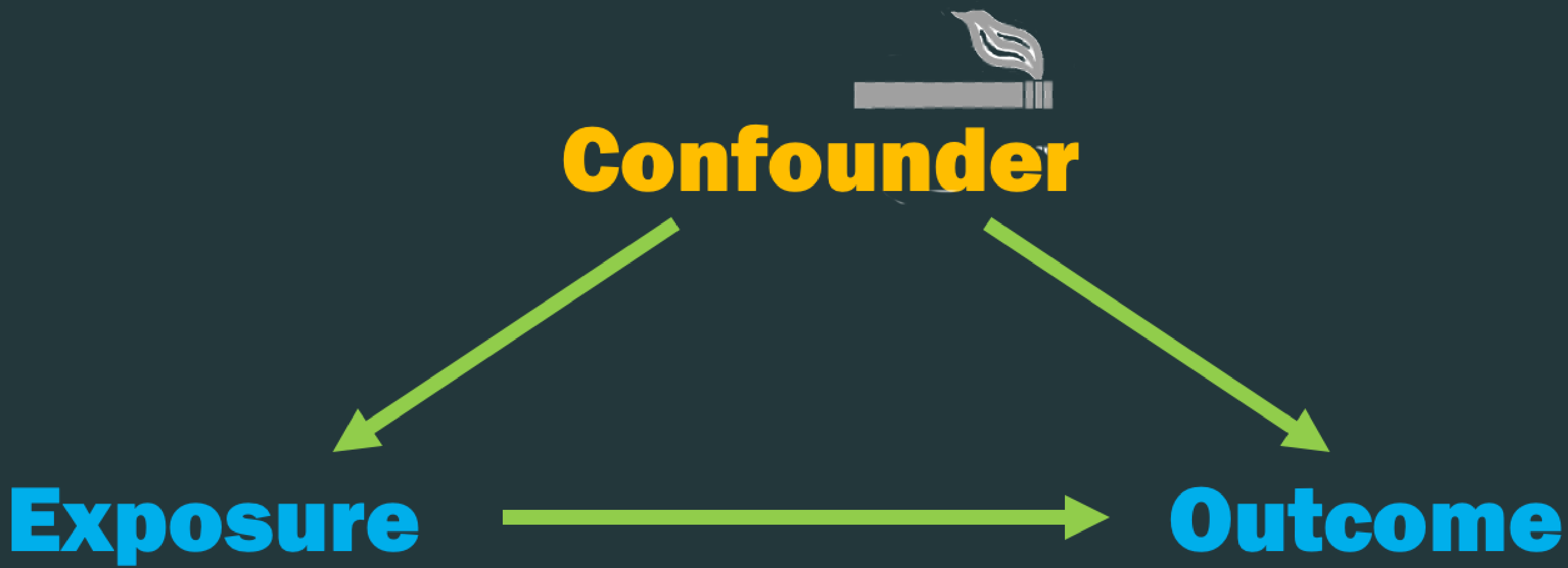
**Control** P=0.5

# Observational Studies

# Confounding



# Confounding



# One binary confounder

# Simulation

```r
n <- 1000
sim <- tibble(
  confounder = rbinom(n, 1, 0.5),
  p_exposure = case_when(
    confounder == 1 ~ 0.75,
    confounder == 0 ~ 0.25
  ),
  exposure = rbinom(n, 1, p_exposure),
  # potential outcomes: exposure is not in the
  # outcome equation, so the true effect is 0
  y0 = confounder + rnorm(n),
  y1 = y0,
  # we only ever observe one of the two
  outcome = (1 - exposure) * y0 + exposure * y1
) |>
  select(confounder, exposure, outcome)

sim
```

# Simulation

```
# A tibble: 1,000 × 3
   confounder exposure outcome
        <int>    <int>   <dbl>
 1          0        0  1.13
 2          0        0  1.11
 3          1        1  0.129
 4          1        0  1.21
 5          0        0  0.0694
 6          1        1 -0.663
 7          1        1  1.81
 8          1        1 -0.912
 9          1        0 -0.247
10          0        0  0.998
```

# Simulation

```r
lm(outcome ~ exposure, data = sim)
```

```

Call:
lm(formula = outcome ~ exposure, data = sim)

Coefficients:
(Intercept)     exposure
     0.2688       0.4070
```

- The true difference is 0, because `exposure` does not appear in the outcome equation

# Simulation

```
sim |>
  group_by(exposure) |>
  summarize(avg_y = mean(outcome))
```

```
# A tibble: 2 × 2
  exposure avg_y
     <int> <dbl>
1        0 0.269
2        1 0.676
```

# Simulation

```
sim |>
  group_by(exposure) |>
  summarize(avg_y = mean(outcome)) |>
  pivot_wider(
    names_from = exposure,
    values_from = avg_y,
    names_prefix = "x_"
  ) |>
  summarize(estimate = x_1 - x_0)
```

```
# A tibble: 1 × 1
  estimate
     <dbl>
1    0.407
```

## *Your Turn 1* (`04-ci-with-group-by-and-summarise-exercises.qmd`)

Group the dataset by `confounder` and `exposure`

Calculate the mean of the `outcome` for the groups

# Your Turn 1

```r
sim |>
  group_by(confounder, exposure) |>
  summarize(avg_y = mean(outcome), .groups = "drop")
```

```
# A tibble: 4 × 3
  confounder exposure    avg_y
       <int>    <int>    <dbl>
1          0        0 -0.00907
2          0        1 -0.0166
3          1        0  1.09
4          1        1  0.936
```

# Your Turn 1

```r
sim |>
  group_by(confounder, exposure) |>
  summarize(avg_y = mean(outcome), .groups = "drop") |>
  pivot_wider(
    names_from = exposure,
    values_from = avg_y,
    names_prefix = "x_"
  ) |>
  reframe(estimate = x_1 - x_0) |>
  # note: we would need to weight this
  # if the confounder groups were not equal sized
  summarize(estimate = mean(estimate))
```

```
# A tibble: 1 × 1
  estimate
     <dbl>
1  -0.0794
```

🎉

# Two binary confounders

# Simulation

```r
n <- 1000
sim2 <- tibble(
  confounder_1 = rbinom(n, 1, 0.5),
  confounder_2 = rbinom(n, 1, 0.5),
  p_exposure = case_when(
    confounder_1 == 1 & confounder_2 == 1 ~ 0.75,
    confounder_1 == 0 & confounder_2 == 1 ~ 0.9,
    confounder_1 == 1 & confounder_2 == 0 ~ 0.2,
    confounder_1 == 0 & confounder_2 == 0 ~ 0.1,
  ),
  exposure = rbinom(n, 1, p_exposure),
  outcome = confounder_1 + confounder_2 + rnorm(n)
) |>
  select(-p_exposure)

sim2
```

# Simulation

```
# A tibble: 1,000 × 4
   confounder_1 confounder_2 exposure outcome
          <int>        <int>    <int>   <dbl>
 1            0            0        0   0.521
 2            1            0        0   1.38
 3            0            0        0  -0.624
 4            0            1        1   0.427
 5            1            0        1   1.31
 6            0            0        0  -0.707
 7            1            1        1   2.52
 8            1            0        0   1.45
 9            0            0        0  -0.505
10            0            1        1   0.793
# ℹ 990 more rows
```

# Simulation

```r
lm(outcome ~ exposure, data = sim2)
```

```

Call:
lm(formula = outcome ~ exposure, data = sim2)

Coefficients:
(Intercept)     exposure  
     0.6395       0.6951  
```

- The true difference is again 0, because `exposure` does not appear in the outcome equation

# *Your Turn 2*

**Group the dataset by the confounders and exposure**

**Calculate the mean of the outcome for the groups**

# Your Turn 2

```r
sim2 |>
  group_by(confounder_1, confounder_2, exposure) |>
  summarize(avg_y = mean(outcome), .groups = "drop") |>
  pivot_wider(
    names_from = exposure,
    values_from = avg_y,
    names_prefix = "x_"
  ) |>
  reframe(estimate = x_1 - x_0) |>
  summarize(estimate = mean(estimate))
```

```
# A tibble: 1 × 1
  estimate
     <dbl>
1  -0.0731
```

# Simulation

```r
n <- 100000
big_sim2 <- tibble(
  confounder_1 = rbinom(n, 1, 0.5),
  confounder_2 = rbinom(n, 1, 0.5),
  p_exposure = case_when(
    confounder_1 == 1 & confounder_2 == 1 ~ 0.75,
    confounder_1 == 0 & confounder_2 == 1 ~ 0.9,
    confounder_1 == 1 & confounder_2 == 0 ~ 0.2,
    confounder_1 == 0 & confounder_2 == 0 ~ 0.1,
  ),
  exposure = rbinom(n, 1, p_exposure),
  outcome = confounder_1 + confounder_2 + rnorm(n)
) |>
  select(-p_exposure)

big_sim2
```

# Simulation

```
# A tibble: 100,000 × 4
   confounder_1 confounder_2 exposure outcome
          <int>        <int>    <int>   <dbl>
 1            1            1        1   2.35 
 2            1            1        0   3.71 
 3            0            0        0   2.08 
 4            0            1        1   0.516
 5            0            0        0  -0.166
 6            1            1        1   1.58 
 7            0            0        0   0.472
 8            1            0        0   3.22 
 9            0            1        1   0.929
10            0            1        1   1.41 
# ℹ 99,990 more rows
```

# Simulation

```
lm(outcome ~ exposure, data = big_sim2)
```

```

Call:
lm(formula = outcome ~ exposure, data = big_sim2)

Coefficients:
(Intercept)     exposure  
     0.6782       0.6561  
```

# Simulation

```r
big_sim2 |>
  group_by(confounder_1, confounder_2, exposure) |>
  summarize(avg_y = mean(outcome), .groups = "drop") |>
  pivot_wider(
    names_from = exposure,
    values_from = avg_y,
    names_prefix = "x_"
  ) |>
  reframe(estimate = x_1 - x_0) |>
  summarize(estimate = mean(estimate))
```

```
# A tibble: 1 × 1
  estimate
     <dbl>
1   0.0187
```

# Continuous confounder?

# Simulation

```
set.seed(1)
n <- 10000
sim3 <- tibble(
  confounder = rnorm(n),
  p_exposure = exp(confounder) / (1 + exp(confounder)),
  exposure = rbinom(n, 1, p_exposure),
  outcome = confounder + rnorm(n)
) |>
  select(-p_exposure)

sim3
```

# Simulation

```
# A tibble: 10,000 × 3
   confounder exposure outcome
        <dbl>    <int>   <dbl>
 1     -0.626        0 -0.840
 2      0.184        1  0.0769
 3     -0.836        0 -1.30
 4      1.60         1  0.911
 5      0.330        1 -0.461
 6     -0.820        0 -1.16
 7      0.487        1 -0.780
 8      0.738        1 -0.656
 9      0.576        1  0.995
10     -0.305        0  2.91
# ℹ 9,990 more rows
```

# Simulation

```r
lm(outcome ~ exposure, data = sim3)
```

```
Call:
lm(formula = outcome ~ exposure, data = sim3)

Coefficients:
(Intercept)     exposure  
    -0.4287       0.8857  
```

- The true difference is still 0, because `exposure` does not appear in the outcome equation

# *Your Turn 3*

**Use `ntile()` from dplyr to calculate a binned version of `confounder` called `confounder_q`. We'll create a variable with 5 bins.**

**Group the dataset by the binned variable you just created and exposure**

**Calculate the mean of the outcome for the groups**

# Your Turn 3

```
sim3 |>
  mutate(confounder_q = ntile(confounder, 5)) |>
  group_by(confounder_q, exposure) |>
  summarize(avg_y = mean(outcome), .groups = "drop") |>
  pivot_wider(
    names_from = exposure,
    values_from = avg_y,
    names_prefix = "x_"
  ) |>
  reframe(estimate = x_1 - x_0) |>
  summarize(estimate = mean(estimate))
```

```
# A tibble: 1 × 1
  estimate
     <dbl>
1    0.127
```

# How many bins?

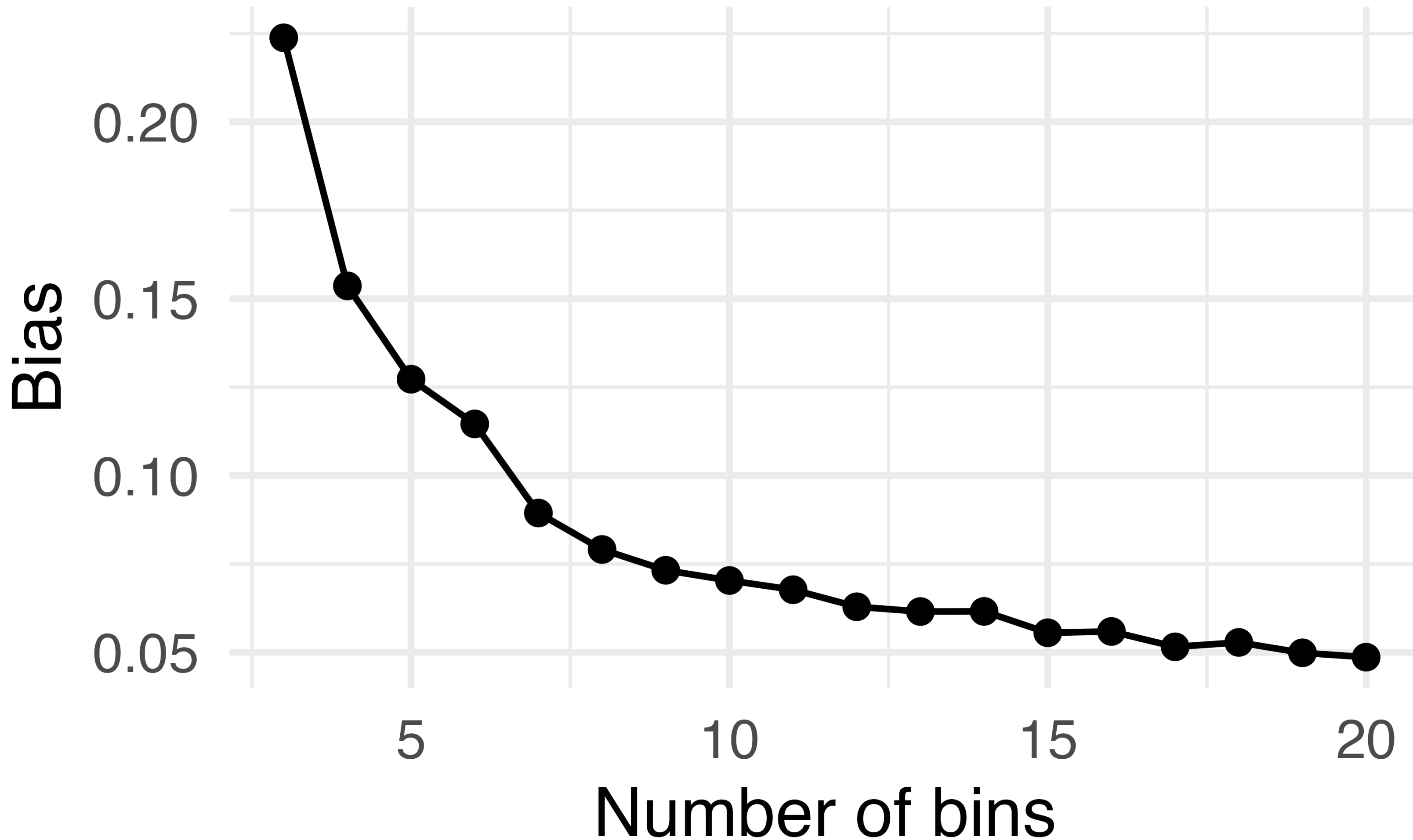
```r
bin_estimate <- function(bins) {
  sim3 |>
    mutate(confounder_q = ntile(confounder, bins)) |>
    group_by(confounder_q, exposure) |>
    summarize(avg_y = mean(outcome), .groups = "drop") |>
    pivot_wider(
      names_from = exposure,
      values_from = avg_y,
      names_prefix = "x_"
    ) |>
    reframe(estimate = x_1 - x_0) |>
    summarize(bins = bins, estimate = mean(estimate))
}
```

# How many bins?

```r
map(3:20, bin_estimate) |>
  bind_rows() |>
  ggplot(aes(x = bins, y = abs(estimate))) +
  geom_point() +
  geom_line() +
  labs(y = "Bias", x = "Number of bins") +
  theme_minimal(base_size = 20)
```

- As we increase the number of bins, the bias decreases

# How many bins?



# How many bins?

```r
bin_estimate(500)
```

```
# A tibble: 1 × 2
   bins estimate
  <dbl>    <dbl>
1   500       NA
```

- The estimate is NA

# A positivity violation

- Some bins have no one in one of the exposure groups, making their difference inestimable

- The violation is stochastic: it comes from our sample size, 10,000, and the number of bins, 500

# A parametric alternative

- This non-parametric method, while flexible, is limited by our sample size
- Parametric models let us extrapolate under certain assumptions

# A parametric alternative

```r
lm(outcome ~ exposure + confounder, data = sim3)
```

```

Call:
lm(formula = outcome ~ exposure + confounder, data = sim3)

Coefficients:
(Intercept)     exposure   confounder  
  0.0002756    0.0392072    0.9892579  
```

- We get the right answer without binning, at the price of the correct **functional form**

# Stratification

- Grouping by confounders is called **stratification**, a **non-parametric** approach

- We aren't using a statistical model to restrict the form of the variables

# The curse of dimensionality

- With many confounders, especially continuous ones, we have too few observations within combinations of confounder levels

- We saw a version of this with 500 bins: no data means no estimate

# What if we could come up with a summary score of all confounders?